

DOCUMENTACIÓN TÉCNICA

Proxy inverso L7 de alto rendimiento C11 · epoll / kqueue

Arquitectura, módulos, protocolos, decisiones de diseño, pruebas y rendimiento.

Versión	1.0.0 — rama <code>feat/proxy-l7</code> (MR!1)
Fecha	26 de septiembre de 2026
Plataformas	Linux (epoll) verificada · macOS/BSD (kqueue) implementada, sin ejecutar
Documentos	SPEC.md · docs/requerimientos.md · docs/decisiones.md · docs/verificacion.md

Índice

1. Visión general

- Alcance
- Cifras clave

2. Arquitectura

- Procesos
- Flujo de una petición
- Estructura del código

3. Event loop (`io_event`)

- API
- Edge-triggered y `sess_drive`
- Timers y reloj
- Liberación diferida

4. Master y workers

- Canal master → worker
- Sockets de escucha
- Señales, relanzamiento y parada

5. Ciclo de vida de una conexión

- Máquina de estados
- Buffers y contrapresión
- Timeouts

6. HTTP/1.1

- Parser
- Reescritura de cabeceras
- Cuerpos
- Keep-alive, pool y reintento
- Pipelining y WebSocket
- Errores generados por el proxy

7. TLS y SNI

8. Enrutamiento, balanceo y salud

9. Configuración y recarga en caliente

10. Observabilidad

11. Pruebas y rendimiento

12. Decisiones de diseño y limitaciones

1. Visión general

El proyecto es un **proxy inverso de capa 7** escrito en C11 sin frameworks, directamente sobre las APIs del sistema operativo. Recibe conexiones HTTP y HTTPS en uno o varios puertos, decide el destino por el nombre de dominio (Host:) y reparte las peticiones entre varios servidores backend.

1.1 Alcance

Incluido	Fuera de alcance
Linux (<code>epoll</code>) y macOS/BSD (<code>kqueue</code>)	Windows / IOCP
HTTP/1.0 y HTTP/1.1 en ambos lados	HTTP/2 y HTTP/3
Terminación TLS (OpenSSL $\geq$ 3.0) con SNI	TLS hacia los backends
Keep-alive, pipelining, chunked, WebSocket	Caché y compresión
Recarga automática y por SIGHUP	API de administración remota

1.2 Cifras clave

223.705 req/s HTTPS (100 conexiones)	337.904 req/s HTTP (200 conexiones)	0 errores en 28,5 M peticiones	96 + 31 comprobaciones de integración + tests unitarios
--	---	--	---

La meta del enunciado ($\geq$ 50.000 req/s sobre HTTPS) se supera entre 3,4× y 4,5× en todos los escenarios medidos (sección 11).

2. Arquitectura

2.1 Procesos

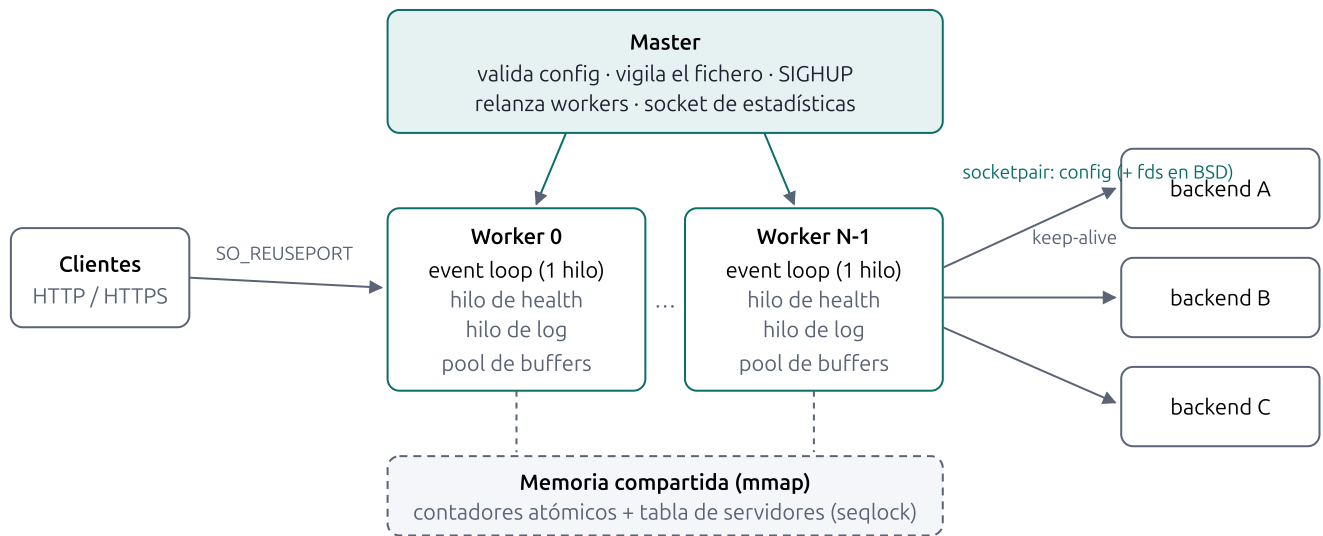


Figura 1. Master, workers y memoria compartida. Cada worker es un proceso independiente con un único hilo de event loop.

El **master** no atiende tráfico. Valida la configuración, crea los workers con `fork()`, les envía la config por un `socketpair`, vigila el fichero, atiende las señales, relanza workers caídos y sirve las estadísticas. Cada **worker** ejecuta un event loop de un solo hilo y dos hilos auxiliares (sondas de salud y escritura del log) que nunca tocan las conexiones.

2.2 Flujo de una petición

1. El kernel entrega la conexión a un worker (`SO_REUSEPORT` en Linux).
2. Si el frontend es TLS, se completa el handshake y se elige el certificado por SNI.
3. Se parsea la cabecera en cuanto está completa y se busca la ruta por `Host: : exacta` → `wildcard` → `default`.
4. La estrategia del backend elige un servidor sano. Se reutiliza una conexión del pool keep-alive o se abre una nueva.
5. Se reenvía la cabecera reescrita (`X-Forwarded-*`, sin hop-by-hop) y el cuerpo en streaming.
6. La respuesta vuelve reescrita (sin `X-Backend-Load`). Al terminar, la conexión al backend vuelve al pool y la del cliente espera la siguiente petición.

2.3 Estructura del código

Módulo	Responsabilidad
<code>io_event_*</code>	Event loop sobre epoll/kqueue, timers y callbacks diferidos
<code>master</code> , <code>main</code>	Arranque, workers, recarga, señales y socket de estadísticas
<code>worker</code>	Loop del worker, accept, swap de generaciones y parada ordenada
<code>listener</code>	Sockets de escucha y canal master→worker (paso de fds con <code>SCM_RIGHTS</code>)
<code>connection</code>	Máquina de estados cliente↔upstream, pool keep-alive, reintentos y timeouts
<code>http_parser</code>	Parser HTTP/1.x incremental y delimitación de cuerpos
<code>tls</code>	Contextos OpenSSL por certificado y callback de SNI
<code>router</code>	Tabla hash djb2 de dominios exactos, wildcards por sufijo más largo, default
<code>backend_pool</code>	Servidores, estrategias de balanceo y estado de salud atómico
<code>health</code>	Sondas TCP/HTTP en un hilo por generación
<code>runtime</code>	Generación de config "compilada" con refcount (RCU)
<code>config</code>	Parser TOML (tomlc99) con validación estricta
<code>watch_*</code>	Vigilancia del fichero: inotify / <code>EVFILT_VNODE</code>
<code>log</code>	Ring buffer MPSC de 4096 × 512 B y hilo consumidor
<code>stats</code>	Memoria compartida y JSON agregado
<code>buffer_pool</code>	Arena <code>mmap</code> de slots de 16 KB con freelist

Todo salvo `main` , `master` , `worker` , `runtime` , `listener` y `connection` se compila como la biblioteca estática `proxycore` , que reutilizan los tests unitarios y las herramientas `test_backend` y `ws_probe` .

3. Event loop (`io_event`)

3.1 API

```
io_loop_t *io_loop_create(int max_events);
int io_loop_add(io_loop_t *, int fd, uint32_t events, io_handler_t *h);
int io_loop_mod(io_loop_t *, int fd, uint32_t events, io_handler_t *h);
int io_loop_del(io_loop_t *, int fd);
int io_loop_run(io_loop_t *);          /* hasta io_loop_stop() */
void io_loop_stop(io_loop_t *);
void io_loop_destroy(io_loop_t *);
void io_loop_abandon(io_loop_t *);    /* tras fork(): no toca el kernel */

void io_timer_set(io_loop_t *, io_timer_t *, uint64_t deadline_ms);
void io_timer_cancel(io_loop_t *, io_timer_t *);
void io_loop_defer(io_loop_t *, void (*fn)(void *), void *arg);
/* eventos: IO_READ, IO_WRITE, IO_ERROR, IO_HUP */
```

Cada objeto registrado embebe un `io_handler_t` con su callback. `io_event_epoll.c` y `io_event_kqueue.c` implementan solo `add/mod/del` y la espera; los timers y los callbacks diferidos son comunes (`io_event_common.c`).

3.2 Edge-triggered y `sess_drive`

Todos los fds se registran una sola vez, con lectura y escritura, en modo **edge-triggered** (`EPOLLET` / `EV_CLEAR`). El kernel avisa una vez por cambio de estado, así que cada sesión lleva dos flags por fd: `rd` y `wr` ("el último intento no terminó en `EAGAIN` ").

```
sess_drive(s):
  repetir (máx. 32 vueltas):
    cli_read      /* socket cliente → in (hasta EAGAIN o buffer lleno) */
    process       /* parsear, enrutar, mover cuerpo in → uo          */
    up_io         /* connect, uo → backend, backend → ui            */
    process       /* respuesta ui → out                             */
    cli_write     /* out → socket cliente                            */
  mientras haya progreso
  si se agota el presupuesto: io_loop_defer(redrive)
```

Como se repite hasta que no hay progreso, nunca queda trabajo pendiente sin un evento que lo despierte, el error clásico con edge-triggered. El presupuesto de 32 vueltas evita que una transferencia grande acapare el loop.

3.3 Timers y reloj

Los timers viven en un min-heap. `io_timer_set` es **perezoso**: retrasar un deadline solo actualiza un campo, y el timer se reubica cuando vence su clave antigua; adelantarlo sí reordena el heap. Así, refrescar un timeout con cada byte recibido cuesta $O(1)$.

Bug corregido. `io_loop_now()` es un reloj cacheado por iteración. En la primera versión se actualizaba *después* de despachar los eventos, así que tras un periodo ocioso los handlers calculaban deadlines ya vencidos (408/504 espurios). Ahora se actualiza justo después de `epoll_wait` / `kevent`, con un test de regresión (`test_now_fresh_in_handler`).

3.4 Liberación diferida

Un mismo lote de `epoll_wait` puede traer varios eventos de un objeto que se cierra al procesar el primero. Liberarlo en ese momento sería un use-after-free. Por eso `close` marca el objeto como cerrado (los eventos tardíos se ignoran) y el `free` se encola con `io_loop_defer`, que se ejecuta al final de la iteración en orden FIFO.

4. Master y workers

4.1 Canal master → worker

Cada worker recibe su configuración por un `socketpair`. El mensaje lleva una cabecera fija y, en BSD, descriptores adjuntos (`SCM_RIGHTS`):

```
{ magic = 0x50525859, cmd = CHAN_CONFIG, len, nfd }
payload: base_dir '\0' texto_toml '\0' claves_de_listener separadas por '\n'
```

El master envía el **texto** ya validado en lugar de pedir a cada worker que relea el fichero. Así todos aplican exactamente la misma versión aunque el fichero cambie mientras tanto. Si el canal se cierra (murió el master), el worker inicia una parada ordenada: no quedan procesos huérfanos.

4.2 Sockets de escucha

Plataforma	Estrategia	Motivo
Linux	Cada worker abre sus sockets con <code>SO_REUSEPORT</code>	El kernel reparte las conexiones por hash, sin <i>thundering herd</i>
macOS/BSD	El master abre los sockets y los envía a los workers por el canal	Allí <code>SO_REUSEPORT</code> no reparte carga

En una recarga, cada worker sincroniza sus listeners con los frontends nuevos: conserva los existentes, abre los añadidos y cierra los eliminados.

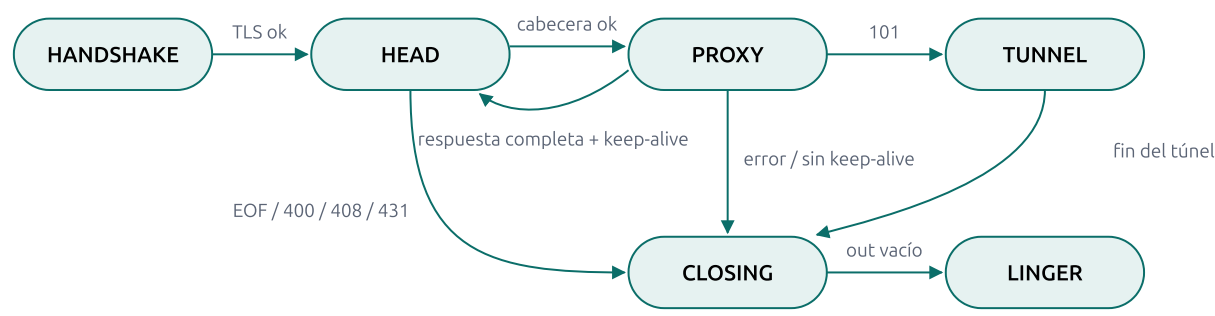
4.3 Señales, relanzamiento y parada

Evento	Master	Worker
<code>SIGHUP</code>	Recarga forzada	Ignorada (la coordina el master)
Cambio en el fichero	Recarga tras 200 ms de debounce, solo si el contenido cambió	—
<code>SIGTERM</code> / <code>SIGINT</code>	Reenvía <code>SIGTERM</code> , espera 12 s y luego <code>SIGKILL</code>	Cierra listeners y conexiones ociosas; espera hasta 10 s a las activas
<code>SIGCHLD</code>	Relanza el worker con backoff; abandona tras 10 fallos seguidos al arrancar	—

Todas las señales se procesan con el *self-pipe trick*: el handler solo escribe un byte en una tubería registrada en el loop.

5. Ciclo de vida de una conexión

5.1 Máquina de estados



LINGER: shutdown(SHUT_WR) y descartar la entrada hasta EOF o 2 s, para que un RST no se coma la respuesta.

Figura 2. Estados de una sesión de cliente (connection.c).

5.2 Buffers y contrapresión

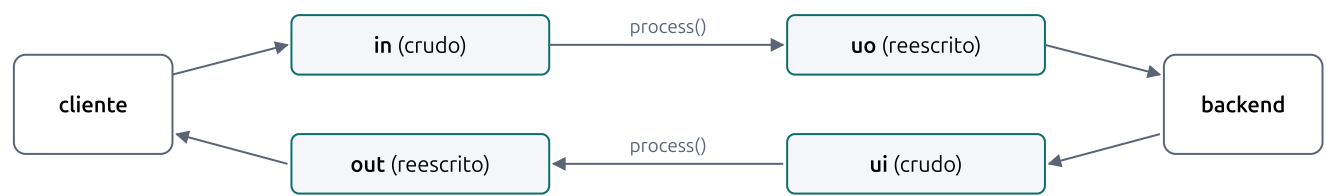


Figura 3. Los cuatro buffers de 16 KB de una sesión.

Los buffers salen de un pool mmap por worker y se piden solo cuando hacen falta: una conexión keep-alive ociosa no retiene ninguno. La contrapresión sale sola. Si out se llena porque el cliente lee despacio, process deja de vaciar ui, se deja de leer del backend y la ventana TCP frena al servidor. Con 10 MB de subida, el pico de memoria medido por worker fue de 13 MB.

5.3 Timeouts

Estado	Deadline	Al vencer
HANDSHAKE	aceptación + client_header	cerrar
HEAD, sin datos	entrada en el estado + client_idle	cerrar (keep-alive ocioso)
HEAD, cabecera a medias	primer byte + client_header	408
PROXY, conectando	inicio del connect + upstream_connect	fallo pasivo; otro servidor, o 504
PROXY	último progreso + upstream_read	504 (o cerrar si ya hay respuesta a medias)
TUNNEL	último progreso + client_idle	cerrar
CLOSING / LINGER	5 s / 2 s	cerrar

6. HTTP/1.1

6.1 Parser

La cabecera se busca como `\r\n\r\n`, recordando hasta dónde se ha escaneado, y se parsea de una vez cuando está completa. Los campos son punteros al buffer, sin copias. Así se tolera cualquier fragmentación (hay tests que alimentan el parser byte a byte y en trozos aleatorios) sin reescanear. Validaciones:

- Método como *token*, versión exactamente `HTTP/1.0` o `HTTP/1.1`, un único espacio entre elementos.
- Nombres de cabecera como *token*, sin espacio antes de `:`. Se rechazan el *obs-fold* y los caracteres de control.
- Host obligatorio en 1.1 y único; se le quita el puerto.
- **Anti request smuggling:** `Content-Length` + `Transfer-Encoding` → 400; varios CL distintos → 400; TE sin `chunked` final → 400.
- Cabecera de más de 16 KB → 431.

6.2 Reescritura de cabeceras

Hacia el backend	Hacia el cliente
Línea de petición reescrita a <code>HTTP/1.1</code>	Sin hop-by-hop ni <code>X-Backend-Load</code>
<code>X-Forwarded-For</code> : se añade la IP (concatenando)	<code>Connection: close</code> si se va a cerrar
<code>X-Forwarded-Proto</code> y <code>X-Real-IP</code> : se reemplazan	<code>Connection: keep-alive</code> para clientes
Sin hop-by-hop (<code>Connection</code> , <code>Keep-Alive</code> , TE , <code>Trailer</code> , <code>Upgrade</code> ... y las nombradas en <code>Connection</code>)	1.0 persistentes
	En un 101 se conservan <code>Connection</code> y <code>Upgrade</code>

Nunca se eliminan `Host` , `Content-Length` ni `Transfer-Encoding` , aunque aparezcan en `Connection: :` si se pudiera forzar al proxy a quitar el encuadre, eso sería un vector de smuggling.

6.3 Cuerpos

`http_body_t` solo sigue los límites del mensaje. Los bytes se reenvían tal cual, sin decodificar ni almacenar.

Modo	Cuándo
NONE	Sin cuerpo; <code>HEAD</code> , <code>1xx</code> , <code>204</code> , <code>304</code>
LENGTH	<code>Content-Length</code>
CHUNKED	Máquina de estados de 9 estados: tamaño hex, extensiones, datos, CRLF y trailers
EOF	Respuesta sin longitud: se reenvía hasta el cierre y después se cierra también el cliente

6.4 Keep-alive, pool y reintento

- **Cliente:** persistente por defecto en 1.1; en 1.0 solo con `Connection: keep-alive`.
- **Backend:** pool LIFO de conexiones ociosas por servidor (`max_idle` , `upstream_idle`). Una conexión vuelve al pool si la respuesta terminó limpia, el backend no pidió cerrar y no sobran bytes. Mientras está en el pool, cualquier dato o cierre la invalida (se confirma con `MSG_PEEK` para descartar eventos tardíos).
- **Reintento único:** si una conexión *reutilizada* falla antes del primer byte de respuesta y la petición es idempotente y sin cuerpo, se reenvía por una conexión nueva. Para eso se conserva la cabecera ya enviada en `uo`. Un POST nunca se reintenta.

- Si el connect falla, se prueba con otro servidor (hasta 3 intentos, excluyendo los ya probados).

6.5 Pipelining y WebSocket

Las peticiones pipelined se leen por adelantado hasta llenar `in`, pero se despachan en serie: la siguiente empieza cuando la respuesta anterior se ha leído entera del backend. Así el orden queda garantizado sin colas, y cada petición puede ir a un backend distinto.

Si la petición trae `Connection: Upgrade` y el backend responde `101`, la sesión pasa a **TUNNEL**: copia bytes en ambos sentidos hasta que un lado cierra. La conexión nunca vuelve al pool.

6.6 Errores generados por el proxy

Código	Causa
400	Petición malformada, smuggling, chunked inválido, 1.1 sin <code>Host</code>
404	Ninguna ruta (ni <code>default</code>) para el <code>Host</code>
408	Cabecera incompleta tras <code>client_header</code>
421	SNI y <code>Host</code> distintos, y el certificado servido no cubre el <code>Host</code>
431	Cabecera de más de 16 KB
502	Backend inalcanzable tras los intentos, respuesta inválida o conexión rota
503	Ningún servidor sano en el backend, o frontend eliminado en una recarga
504	Sin datos del backend durante <code>upstream_read</code>

Todas estas respuestas llevan `Connection: close`.

7. TLS y SNI

Cada certificado de un frontend tiene su `SSL_CTX` (TLS $\geq$ 1.2, sin compresión ni renegociación, escrituras parciales). El SSL nace con el contexto del `default_cert`, y el callback de servername lo cambia con `SSL_set_SSL_CTX` según este orden:

1. Coincidencia exacta con el patrón `sni` de la config.
2. Wildcard de **una sola etiqueta** (RFC 6125): `*.example.com` cubre `a.example.com`, no `a.b.example.com`. Gana el patrón más largo.
3. Si no hay coincidencia, `default_cert`.

Tras el handshake, la sesión copia el nombre SNI y el patrón servido. Si el cliente envió SNI, el `Host` es distinto y el patrón no lo cubre $\rightarrow$ `421 Misdirected Request`. El handshake no bloquea: `SSL_ERROR_WANT_READ/WRITE` se traducen a los flags `rd / wr`, incluidos los casos cruzados (una lectura que necesita escribir, y al revés).

8. Enrutamiento, balanceo y salud

8.1 Enrutamiento

Cada frontend tiene su tabla: hash djb2 con direccionamiento abierto para dominios exactos, lista de wildcards ordenada por longitud de sufijo (el más largo gana; aquí sí cubren varias etiquetas) y ruta `default` opcional. Todas las búsquedas son insensibles a mayúsculas.

8.2 Estrategias de balanceo

Estrategia	Algoritmo
<code>round_robin</code>	Siguiente servidor sano en orden
<code>weighted</code>	<i>Smooth weighted round robin</i> (nginx): con pesos 3:1 reparte 75/25 sin ráfagas largas
<code>least_conn</code>	Menos peticiones activas en este worker; si empatan, rotación
<code>least_load</code>	Carga de <code>X-Backend-Load</code> suavizada con EMA ($\alpha = 0,3$), que caduca a los 5 s. Se eligen dos candidatos al azar (<i>power of two choices</i>) y gana el de menor carga. Si no hay carga fresca, decide <code>least_conn</code> ; si también empatan, gana el que no tiene carga conocida (exploración)

8.3 Salud

- **Pasiva** (hilo del loop): connect rechazado o reset antes de la respuesta cuentan como fallo; tras `fall` fallos seguidos, el servidor pasa a `down`. Sin sondas activas, pasado `interval` se reintenta en modo *half-open*: basta un fallo para volver a `down`.
- **Activa** (hilo propio por generación): TCP connect o `GET path` esperando `2xx`, cada `interval`; tras `rise` éxitos, `up`.
- El estado (`up`, `fails`, `oks`) es atómico: es lo único que comparten los dos hilos.

9. Configuración y recarga en caliente

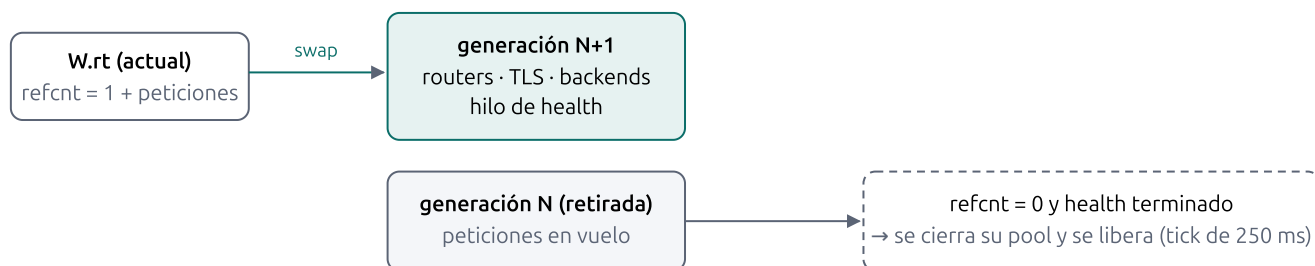


Figura 4. Generaciones de configuración con refcount (RCU sin atómicos: todo ocurre en el hilo del loop).

1. **Disparo:** inotify sobre el *directorio* (cubre la escritura in situ y el rename de `vim / sed -i`), `EVFILT_VNODE` sobre fichero y directorio en `kqueue`, o `SIGHUP`. Debounce de 200 ms. Si el contenido no cambió, no se hace nada (salvo con `SIGHUP`).
2. **Validación en el master:** TOML, reglas de coherencia y carga de certificados. Si falla, se registra el error y se mantiene la configuración actual.
3. **Envío** del texto a cada worker, que construye la nueva generación, sincroniza sus listeners y cambia el puntero.
4. Las peticiones en vuelo terminan con su generación; en las conexiones keep-alive, la petición siguiente ya usa la nueva.

La validación es estricta: una clave desconocida (`workerz`) es un error. Así una errata no se ignora en silencio.

10. Observabilidad

10.1 Log

Cada proceso tiene un ring buffer MPSC sin locks (cola acotada de Vyukov, 4096×512 B). El event loop y el hilo de health escriben en él, y un hilo consumidor vuelca bloques de hasta 64 KB con `write()` en `O_APPEND`, que no parte las líneas entre procesos. Si el ring se llena, el mensaje se descarta y se incrementa `log_dropped`: el loop nunca se bloquea.

10.2 Estadísticas

El master crea una región `mmap(MAP_SHARED)` antes del `fork`. Cada worker incrementa sus contadores atómicos (*relaxed*, cada uno en su línea de caché) y cada 250 ms publica su tabla de servidores con un seqlock. El master agrega todo y sirve JSON por el socket UNIX `stats_socket`. Un servidor aparece como `degraded` cuando los workers discrepan sobre su salud.

11. Pruebas y rendimiento

11.1 Estrategia de pruebas

Nivel	Contenido	Casos
Unitario (cmocka)	Parser (fragmentación, límites, smuggling, chunked), router, balanceo (4 estrategias, salud), config (17 reglas), núcleo (pool, timers, reloj, ring de log concurrente)	31
Integración	12 backends reales y el proxy: enrutamiento, errores, cabeceras, cuerpos de 10 MB, keep-alive, pool, reintento, pipelining, timeouts, WebSocket, TLS/SNI, balanceo, failover, recarga, stats, relanzamiento, carga con recargas y parada	96
Sanitizers	Todo lo anterior con AddressSanitizer + UBSan + LeakSanitizer	0 informes

Herramientas propias: `test_backend` (HTTP/1.1 y WebSocket sobre `io_event`, con latencia, carga y descarte de conexiones configurables) y `ws_probe` (cliente WebSocket en C, con TLS).

11.2 Benchmark

Escenario (wrk, 30 s)	Protocolo	Req/s	Latencia media	p99	Errores
-t4 -c100	HTTPS	223.705	0,43 ms	1,29 ms	0
-t4 -c200	HTTPS	213.565	0,85 ms	2,37 ms	0
-t4 -c400	HTTPS	172.075	1,97 ms	4,78 ms	0
-t4 -c200	HTTP	337.904	0,53 ms	1,80 ms	0

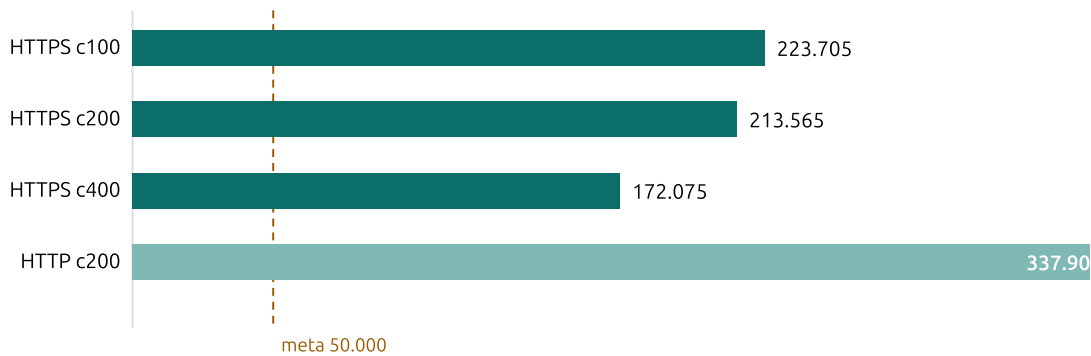


Figura 5. Peticiones por segundo frente a la meta de 50.000.

Entorno: AMD Ryzen 7 7735HS (8 núcleos / 16 hilos) en WSL2, kernel 6.18, OpenSSL 3.5.5. Build release + LTO, 8 workers, access log desactivado, 2 `test_backend` de 2 hilos y respuesta de 100 B. Afinidad con `taskset`: wrk en las CPUs 0-3, el proxy en 4-11 y los backends en 12-15. Hubo 28,5 M peticiones, todas 2xx, servidas con solo 816 conexiones a los backends gracias al pool. Como wrk, proxy y backends comparten máquina, las cifras son una cota inferior.

12. Decisiones de diseño y limitaciones

Decisión	Alternativa descartada	Motivo
Procesos (master + workers)	Un proceso con N hilos	Aislamiento de fallos; sin sincronización entre loops
ET con un único registro por fd	LT, o <code>EPOLL_CTL_MOD</code> por cambio de estado	Menos syscalls; <code>sess_drive</code> evita perder eventos
Cabecera completa + delimitación de cuerpo aparte	Parser byte a byte con estado global	Más simple; sin copias ni reescaneo
Copia en buffers de 16 KB	<code>splice()</code> / zero-copy	Con TLS y reescritura de cabeceras los datos pasan por userland igualmente
Pipelining en serie	Varias peticiones en vuelo hacia backends distintos	Orden garantizado sin colas de respuesta
Errores locales con <code>Connection: close</code>	Mantener keep-alive tras un error	Puede quedar un cuerpo a medio leer; cerrar es lo seguro
Reintento solo idempotente, sin cuerpo y en conexión reutilizada	Reintentar cualquier fallo	No duplicar operaciones con efectos
Health por worker	Estado compartido entre procesos	Sin locks entre procesos; se expone <code>degraded</code>
El master envía el texto de la config	Cada worker relee el fichero	Todos aplican la misma versión validada

12.1 Limitaciones conocidas

- El código kqueue (macOS/BSD) está implementado, pero solo se ha comprobado su sintaxis: falta ejecutarlo en un Mac.
- Un cliente HTTP/1.0 recibe tal cual las respuestas chunked del backend.
- Cambiar `workers` requiere reiniciar.
- En una recarga, cada worker lee los certificados del disco en el hilo del loop (milisegundos, y solo al recargar).
- El timeout `upstream_connect` no tiene test automático: en loopback un connect rechazado falla al instante.